# Normalization of the Plant Nursery Sales and Propagation Database

UNF → 1NF → 2NF → 3NF → BCNF
Case Study 23: Plant Nursery Sales and Propagation

## 1. Purpose of Normalization

Normalization organizes relational data to reduce unnecessary redundancy and prevent insertion, update, and deletion anomalies. For this case study, the design is shown step by step from an unnormalized representation to 3NF, followed by a BCNF check.

> **Important:** This is a design/documentation exercise, not a separate set of SQL commands to execute. The final normalized relations are the tables implemented by the project's schema.sql.

## 2. Starting Point: Unnormalized Form (UNF)

Before normalization, imagine that the nursery stored sales, batches, species, growth stages, and stage history in one large record. A sale may contain several sale items, and a propagation batch may contain several stage-history records. These repeating groups make the record unnormalized.

| Sale information | Batch information | Species information | Repeating groups |
|---|---|---|---|
| SaleID | BatchID / BatchCode | SpeciesID | SaleItems { ... } |
| SaleDate | PropagationDate | CommonName | StageHistory { ... } |
| SaleType | InitialQuantity | ScientificName | |
| CustomerName | Notes | Category | |

### Example of the repeating groups

```
Sale 1 → { (Batch 1, quantity 25, price 80), (Batch 3, quantity 10, price 150) }
Batch 1 → { Propagation, Seedling, Young Plant, Mature Plant, Saleable }
```

### Anomalies present in the unnormalized design

| Anomaly | Problem |
|---|---|
| Update | Repeated species or sale information may need multiple updates, risking inconsistent values. |
| Insertion | A new entity may not be independently insertable if the single record requires an associated sale or batch. |
| Deletion | Deleting the only sale/batch associated with an entity could also remove the only stored information about that entity. |

## 3. UNF → First Normal Form (1NF)

1NF requires atomic values and removal of repeating groups. Each sale item becomes a separate row, and each stage-history event becomes a separate row.

| SALE | SALE_ITEM |
|---|---|
| SaleID | SaleItemID |
| SaleDate | SaleID |
| SaleType | BatchID |
| CustomerName | Quantity |
| | UnitPrice |

For example, Sale 1 is represented by two SALE_ITEM rows: (Sale 1, Batch 1, 25, 80.00) and (Sale 1, Batch 3, 10, 150.00). The same principle is used in BATCH_STAGE_HISTORY.

> **What 1NF removes:** the repeating-group/non-atomic-value problem. It does not by itself remove all redundancy anomalies; those are addressed by the later decompositions.

# 4. 1NF → Second Normal Form (2NF)

2NF requires 1NF and requires every non-key attribute to depend on the whole candidate key, not only part of a composite key.

## Partial dependencies in the sales data

If a 1NF sale-line relation is identified by (SaleID, BatchID), sale-level attributes depend only on SaleID, while batch-level attributes depend only on BatchID. These are partial dependencies.

| Functional dependency | Reason |
|---|---|
| SaleID → SaleDate, SaleType, CustomerName | These attributes describe the sale. |
| BatchID → BatchCode, SpeciesID, PropagationDate, InitialQuantity, Notes | These attributes describe the batch. |
| (SaleID, BatchID) → Quantity, UnitPrice | These describe the particular sale line. |

## 2NF decomposition

| Relation | Main attributes |
|---|---|
| SALE | SaleID (PK), SaleDate, SaleType, CustomerName |
| PROPAGATION_BATCH | BatchID (PK), BatchCode, SpeciesID, PropagationDate, InitialQuantity, Notes |
| SALE_ITEM | SaleItemID (PK), SaleID (FK), BatchID (FK), Quantity, UnitPrice |

## Anomalies addressed by 2NF

| Anomaly | Effect |
|---|---|
| Update | Sale-level and batch-level information is stored once rather than repeated across line rows. |
| Insertion | A sale or batch can be represented without duplicating unrelated attributes. |
| Deletion | Deleting a sale item does not erase the sale's descriptive information or the batch's attributes. |

# 5. 2NF → Third Normal Form (3NF)

3NF removes problematic transitive dependencies, where a non-key attribute depends on another non-key attribute rather than directly on the key.

## Propagation-batch transitive dependency

BatchID → SpeciesID
SpeciesID → CommonName, ScientificName, Category

Therefore species details are transitively determined by BatchID. They describe the species, not the batch, so they belong in SPECIES.

## Growth-stage transitive dependency

StageID → StageName, StageOrder, MaxDays. These attributes describe a growth-stage definition, not an individual batch-stage event, so they belong in GROWTH_STAGE.

| Final relation | Attributes |
|---|---|
| SPECIES | species_id (PK), common_name (UNIQUE), scientific_name (UNIQUE), category |
| GROWTH_STAGE | stage_id (PK), stage_name (UNIQUE), stage_order (UNIQUE), max_days |
| PROPAGATION_BATCH | batch_id (PK), batch_code (UNIQUE), species_id (FK), propagation_date, initial_quantity, notes |
| BATCH_STAGE_HISTORY | batch_stage_id (PK), batch_id (FK), stage_id (FK), entered_at, exited_at, quantity_at_entry, quantity_at_exit |
| SALE | sale_id (PK), sale_date, sale_type, customer_name |
| SALE_ITEM | sale_item_id (PK), sale_id (FK), batch_id (FK), quantity, unit_price |

## Anomalies addressed by 3NF

| Anomaly | Effect |
| --- | --- |
| Update | Species and growth-stage definitions are stored once, avoiding repeated updates. |
| Insertion | A species or growth stage can be inserted independently of a particular batch or sale. |
| Deletion | Deleting a batch/history event does not delete the independent species or growth-stage definition. |

## 6. BCNF Check

BCNF is stronger than 3NF. For every non-trivial functional dependency X → Y, X must be a superkey. The final relations are checked using the keys and UNIQUE constraints implemented in the schema.

| Relation | Relevant determinant | BCNF result |
|---|---|---|
| SPECIES | species_id → other attributes; common_name and scientific_name are UNIQUE. | Satisfied: determinants are candidate keys. |
| GROWTH_STAGE | stage_id → other attributes; stage_name and stage_order are UNIQUE. | Satisfied: determinants are candidate keys. |
| PROPAGATION_BATCH | batch_id → other attributes; batch_code is UNIQUE. | Satisfied: determinants are candidate keys. |
| BATCH_STAGE_HISTORY | batch_stage_id → batch_id, stage_id, entered_at, exited_at, quantities. | Satisfied: batch_stage_id is the key. |
| SALE | sale_id → sale_date, sale_type, customer_name. | Satisfied: sale_id is the key. |
| SALE_ITEM | sale_item_id → sale_id, batch_id, quantity, unit_price. | Satisfied: sale_item_id is the key. |

## 7. Final Normalization Conclusion

The nursery database was decomposed from a hypothetical unnormalized design containing repeating sale-item and stage-history groups. 1NF makes values atomic and removes repeating groups. 2NF separates attributes that depend only on part of a composite business key. 3NF separates transitive dependencies such as species details and growth-stage definitions from batch and history data.

> **Conclusion:** Under the functional dependencies and uniqueness constraints identified for this design, the final six-table schema satisfies 3NF and also passes the BCNF check.

## 8. Mapping to the Implemented PostgreSQL Schema

| Normalization result | Implemented table |
|---|---|
| Species entity | species |
| Growth-stage definition | growth_stage |
| Batch entity | propagation_batch |
| Batch stage transitions | batch_stage_history |
| Sale header | sale |
| Sale line items | sale_item |